Documento de Arquitetura de Software
Sistema Runner

Descrição de Arquitetura baseada no modelo conceitual da ISO/IEC/IEEE 42010:2022

Sistema	Sistema Runner
Contexto	Disciplina de Implementação e Integração de Software — Engenharia de Software
Entidade de interesse	Sistema Runner, incluindo os CLIs assinatura e simulador, a aplicação assinador.jar e as integrações necessárias
Versão do documento	1.0
Data	28/04/2026

Histórico de revisão

Versão	Data	Descrição	Responsável
1.0	28/04/2026	Elaboração inicial do documento de arquitetura de software.	Bárbara Nogueira Carvalho da Silveira

Sumário

1. Introdução
2. Identificação da arquitetura descrita
3. Stakeholders, preocupações e interesses arquiteturais
4. Restrições, premissas e drivers arquiteturais
5. Pontos de vista arquiteturais adotados
6. Visões arquiteturais do Sistema Runner
7. Decisões arquiteturais
8. Correspondências, consistência e rastreabilidade
9. Riscos arquiteturais e pontos de atenção
10. Estratégia de verificação e validação da arquitetura
11. Glossário
12. Referências

1. Introdução

1.1 Finalidade do documento

Este documento apresenta a descrição da arquitetura de software do Sistema Runner. A finalidade é registrar, de forma organizada e rastreável, as principais decisões arquiteturais, os elementos do sistema, suas relações, as responsabilidades de cada componente, as visões necessárias para compreensão do sistema e as justificativas que orientam a implementação.

A estrutura adotada segue os conceitos centrais da ISO/IEC/IEEE 42010:2022 para descrição de arquitetura: identificação da entidade de interesse, stakeholders, preocupações, pontos de vista, visões, modelos, correspondências, decisões e justificativas. O documento não substitui a norma, mas organiza a arquitetura do projeto em um formato compatível com suas recomendações.

1.2 Escopo da arquitetura

A arquitetura descrita abrange o Sistema Runner, composto por aplicações de linha de comando desenvolvidas em Go, uma aplicação Java denominada assinador.jar, mecanismos de integração local e HTTP, provisionamento de ambiente Java, gerenciamento do Simulador do HubSaúde, geração de artefatos multiplataforma e pipeline de CI/CD.

O escopo arquitetural inclui a integração entre os CLIs e aplicações Java, a gestão de processos, a comunicação com dispositivo criptográfico via PKCS#11, o armazenamento local de metadados de execução e cache, além da distribuição segura de binários.

1.3 Exclusões de escopo

- Implementação real de assinatura digital criptográfica.
- Implementação real de validação criptográfica de assinatura.
- Integração com autoridades certificadoras.
- Armazenamento persistente de assinaturas.
- Interface gráfica de usuário.
- Autenticação de usuários.
- Geração de certificados digitais.

1.4 Público-alvo

- Estudantes e desenvolvedores responsáveis pela implementação do Sistema Runner.
- Professor ou avaliador da disciplina de Implementação e Integração de Software.
- Integradores interessados em compreender como o Runner executa e integra aplicações Java.
- Futuros mantenedores responsáveis por evoluir o sistema em novas sprints.
- Usuários técnicos que necessitam compreender a instalação, execução e distribuição dos binários.

2. Identificação da arquitetura descrita

2.1 Entidade de interesse

A entidade de interesse é o Sistema Runner. O sistema tem como objetivo facilitar a execução de aplicações Java via linha de comandos, ocultando do usuário detalhes de instalação, configuração, invocação e gerenciamento do ciclo de vida dessas aplicações.

2.2 Missão do sistema

A missão arquitetural do Sistema Runner é oferecer uma camada de integração simples, multiplataforma e automatizada entre o usuário final e aplicações Java relacionadas à Plataforma HubSaúde. O usuário deve operar comandos CLI simples, enquanto o sistema deve cuidar da localização ou provisionamento do ambiente Java, da invocação das aplicações, do tratamento de erros, do gerenciamento de processos e da apresentação dos resultados.

2.3 Sistemas e elementos relacionados

Elemento	Tipo	Descrição arquitetural
Usuário	Ator externo	Pessoa que interage com o Sistema Runner por meio de comandos de terminal.
Sistema Runner	Sistema de software	Conjunto de CLIs, módulos internos, mecanismos de invocação, provisionamento e distribuição.
assinatura	Contêiner / CLI	Aplicação de linha de comandos responsável por receber comandos de assinatura e validação e invocar o assinador.jar.
simulador	Contêiner / CLI	Aplicação de linha de comandos responsável por iniciar, parar e monitorar o Simulador do HubSaúde.
assinador.jar	Contêiner / aplicação Java	Aplicação Java que valida parâmetros e simula criação e validação de assinatura digital. Pode ser invocada em modo local ou servidor HTTP.
Simulador do HubSaúde	Sistema externo	Aplicação Java/Web gerenciada pelo CLI simulador e acessada via HTTP.
Dispositivo de Assinatura Digital	Sistema externo / hardware	Token ou smart card compatível com PKCS#11 para operações criptográficas reais ou simuladas.
GitHub Releases	Serviço externo	Canal de publicação de binários, checksums e artefatos assinados.

Eclipse Temurin / Adoptium	Serviço externo	Fonte prevista para obtenção automática de JDK/JRE compatível.

2.4 Escopo temporal da arquitetura

A arquitetura é planejada para evolução incremental em quatro sprints. A Sprint 1 estabelece a fundação técnica, o CLI básico, testes e CI/CD. A Sprint 2 entrega o fluxo local de assinatura simulada. A Sprint 3 introduz o modo servidor HTTP e a integração com PKCS#11. A Sprint 4 completa o gerenciamento do Simulador do HubSaúde e a distribuição segura dos artefatos.

3. Stakeholders, preocupações e interesses arquiteturais

3.1 Stakeholders identificados

Stakeholder	Interesse principal	Preocupações arquiteturais associadas
Usuário final do CLI	Executar comandos simples para assinar, validar, iniciar e parar aplicações sem conhecer detalhes de Java.	Usabilidade, mensagens claras, compatibilidade multiplataforma, instalação simples, previsibilidade de execução.
Desenvolvedor do projeto	Implementar e manter os CLIs, o assinador.jar e os mecanismos de integração.	Modularidade, testabilidade, organização de pacotes, facilidade de evolução por sprint, automação de build.
Professor / avaliador	Verificar se o projeto atende ao escopo do trabalho prático e às entregas da disciplina.	Rastreabilidade entre requisitos, arquitetura e implementação; completude documental; critérios de aceitação.
Integrador da Plataforma HubSaúde	Usar o Runner para facilitar execução de aplicações relacionadas à integração de dados em saúde.	Confiabilidade, execução previsível, gerenciamento de processos, integração HTTP, portabilidade.

Mantenedor futuro	Evoluir o sistema sem introduzir regressões.	Baixo acoplamento, decisões documentadas, separação de responsabilidades, testes automatizados.
Equipe de segurança / cadeia de suprimentos	Verificar autenticidade e integridade dos binários distribuídos.	Checksums, assinatura com Cosign, GitHub Releases, rastreabilidade de versões.
Ambiente operacional do usuário	Executar binários em Windows, Linux e macOS.	Compatibilidade com SO, detecção de portas, permissões de processo, cache local em diretório do usuário.

3.2 Preocupações arquiteturais consolidadas

ID	Preocupação arquitetural	Descrição
C-01	Portabilidade	O sistema deve funcionar em Windows, Linux e macOS, usando binários pré-compilados e mecanismos compatíveis com cada plataforma.
C-02	Simplicidade de uso	O usuário deve operar o sistema por comandos CLI claros, sem precisar conhecer comandos Java ou detalhes de configuração.
C-03	Integração CLI–Java	Os CLIs em Go devem invocar aplicações Java de forma local e, quando aplicável, por HTTP.
C-04	Gerenciamento de ciclo de vida	O sistema deve iniciar, monitorar, reutilizar e encerrar processos Java, evitando instâncias duplicadas.
C-05	Provisionamento de ambiente	O sistema deve detectar e, quando necessário,

		baixar JDK/JRE compatível para uso próprio.
C-06	Validação e tratamento de erros	Parâmetros inválidos, ausência de arquivos, falhas de Java, porta ocupada e falhas HTTP devem ser tratados com mensagens claras.
C-07	Segurança de artefatos	Binários publicados devem possuir checksums e assinatura para verificação de integridade e autenticidade.
C-08	Testabilidade e CI/CD	Builds, testes e releases devem ser automatizados, reduzindo erros manuais.
C-09	Manutenibilidade	A estrutura de pacotes deve separar responsabilidades e permitir evolução incremental.
C-10	Baixa latência em uso repetido	O modo servidor HTTP deve reduzir o custo de inicialização repetida da JVM em múltiplas chamadas.

4. Restrições, premissas e drivers arquiteturais

4.1 Restrições tecnológicas

ID	Restrição	Impacto arquitetural
R-01	CLIs desenvolvidos em Go 1.25.	Favorece cross-compilation e geração de binários independentes por plataforma.

R-02	assinador.jar desenvolvido em Java 21.	Impõe necessidade de JDK/JRE compatível e integração entre Go e Java.
R-03	Plataformas-alvo: windows/amd64, linux/amd64 e darwin/amd64.	Exige cuidado com paths, extensão .exe, gestão de processos e empacotamento.
R-04	Distribuição via GitHub Releases.	Pipeline deve gerar artefatos versionados, checksums e assinaturas.
R-05	Uso de PKCS#11 para dispositivo criptográfico.	A arquitetura deve isolar a integração criptográfica no assinador.jar e prever tratamento de ausência de dispositivo.
R-06	Simulação de assinatura, não assinatura real.	Permite focar em validação de parâmetros, fluxo de integração e resposta simulada.

4.2 Premissas arquiteturais

- O usuário terá permissão para executar binários locais e iniciar processos Java em sua máquina.
- O diretório local ~/.hubsaude/ poderá ser usado para cache, metadados de processo, JDK/JRE gerenciado e artefatos baixados.
- O repositório usará a branch principal main e versionamento semântico para releases.
- Os fluxos de assinatura e validação serão simulados, mas deverão preservar formato de entrada, validação e estrutura de saída compatíveis com o uso esperado.
- A comunicação HTTP será usada quando o assinador.jar ou o Simulador estiver em execução como serviço local.
- Os binários gerados no CI/CD serão a forma preferencial de distribuição para usuários finais.

4.3 Drivers arquiteturais

Driver	Motivação	Consequência na arquitetura
Integração simples	Usuários não devem executar comandos Java manualmente.	Criação de CLIs que encapsulam a execução das aplicações Java.
Portabilidade	O sistema deve funcionar nas três principais plataformas desktop.	Escolha de Go para CLI e pipeline de cross-compilation.
Evolução incremental	O trabalho está planejado em quatro sprints.	Separação de módulos e stubs para permitir crescimento controlado.
Confiabilidade operacional	Processos Java precisam ser iniciados, monitorados e encerrados corretamente.	Registro de PID/porta, health check HTTP e diretório de estado local.
Segurança de distribuição	Usuários devem verificar que binários não foram adulterados.	Geração de SHA256 e assinatura Cosign nos releases.

5. Pontos de vista arquiteturais adotados

Um ponto de vista arquitetural define quais preocupações serão tratadas e quais modelos serão usados para representar uma visão da arquitetura. Para este documento, foram adotados pontos de vista adequados ao tamanho e ao contexto do Sistema Runner, combinando C4, decomposição lógica, implantação, integração, segurança e CI/CD.

ID	Ponto de vista	Preocupações atendidas	Modelos/artefatos usados
VP-01	Contexto	Fronteira do sistema, atores externos, sistemas externos e responsabilidades gerais.	Diagrama de Contexto C4 e tabela de atores/sistemas externos.
VP-02	Contêineres	Principais aplicações executáveis,	Diagrama de Contêineres C4 e

		protocolos e responsabilidades.	tabela de comunicação.
VP-03	Decomposição lógica	Organização interna do repositório, pacotes e responsabilidades.	Tabela de módulos, estrutura de diretórios e dependências.
VP-04	Integração e execução	Fluxos CLI–Java, invocação local, HTTP, gerenciamento de processos e tratamento de erro.	Sequências textuais, tabelas de interfaces e fluxos de execução.
VP-05	Implantação e distribuição	Ambientes, binários por plataforma, JDK/JRE, GitHub Releases.	Tabela de ambientes e pipeline de build/release.
VP-06	Segurança	Integridade dos artefatos, PKCS#11, limites de escopo criptográfico.	Tabela de ameaças/controles e decisões de segurança.
VP-07	Qualidade e evolução	Testabilidade, manutenibilidade, portabilidade, desempenho e riscos.	Matriz de requisitos de qualidade, trade-offs e validação arquitetural.

6. Visões arquiteturais do Sistema Runner

6.1 Visão de contexto

A visão de contexto delimita o Sistema Runner em relação aos atores e sistemas externos. O usuário interage por CLI. O Runner invoca ou gerencia aplicações Java e, quando necessário, interage indiretamente com dispositivo de assinatura digital compatível com PKCS#11.

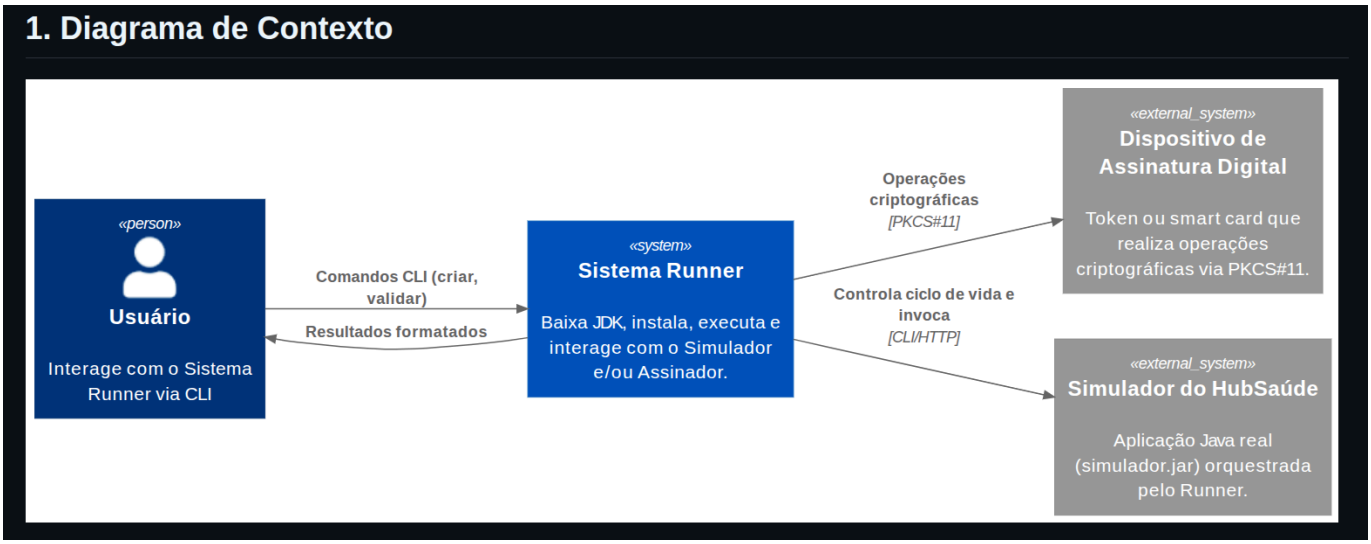

Figura 1 — Diagrama de Contexto do Sistema Runner.

Relação externa	Descrição	Mecanismo
Usuário → Sistema Runner	Usuário envia comandos para criar/validar assinaturas ou gerenciar simulador.	CLI
Sistema Runner → Dispositivo de Assinatura Digital	O assinador.jar interage com token ou smart card para operações criptográficas ou simulações relacionadas.	PKCS#11
Sistema Runner → Simulador do HubSaúde	O CLI simulador inicia, monitora e encerra a aplicação simulador.jar.	CLI/HTTP
Sistema Runner → GitHub Releases	O sistema obtém ou publica binários, checksums e artefatos versionados.	HTTPS
Sistema Runner → Adoptium/Temurin	O sistema baixa JDK/JRE compatível quando ausente.	HTTPS

6.2 Visão de contêineres

A visão de contêineres mostra as principais unidades executáveis e suas formas de comunicação. O Sistema Runner é composto por CLIs em Go e por uma

aplicação Java. O CLI assinatura é responsável pelo fluxo de assinatura; o CLI simulador é responsável pelo gerenciamento do Simulador do HubSaúde; o assinador.jar executa a validação e simulação das operações de assinatura.

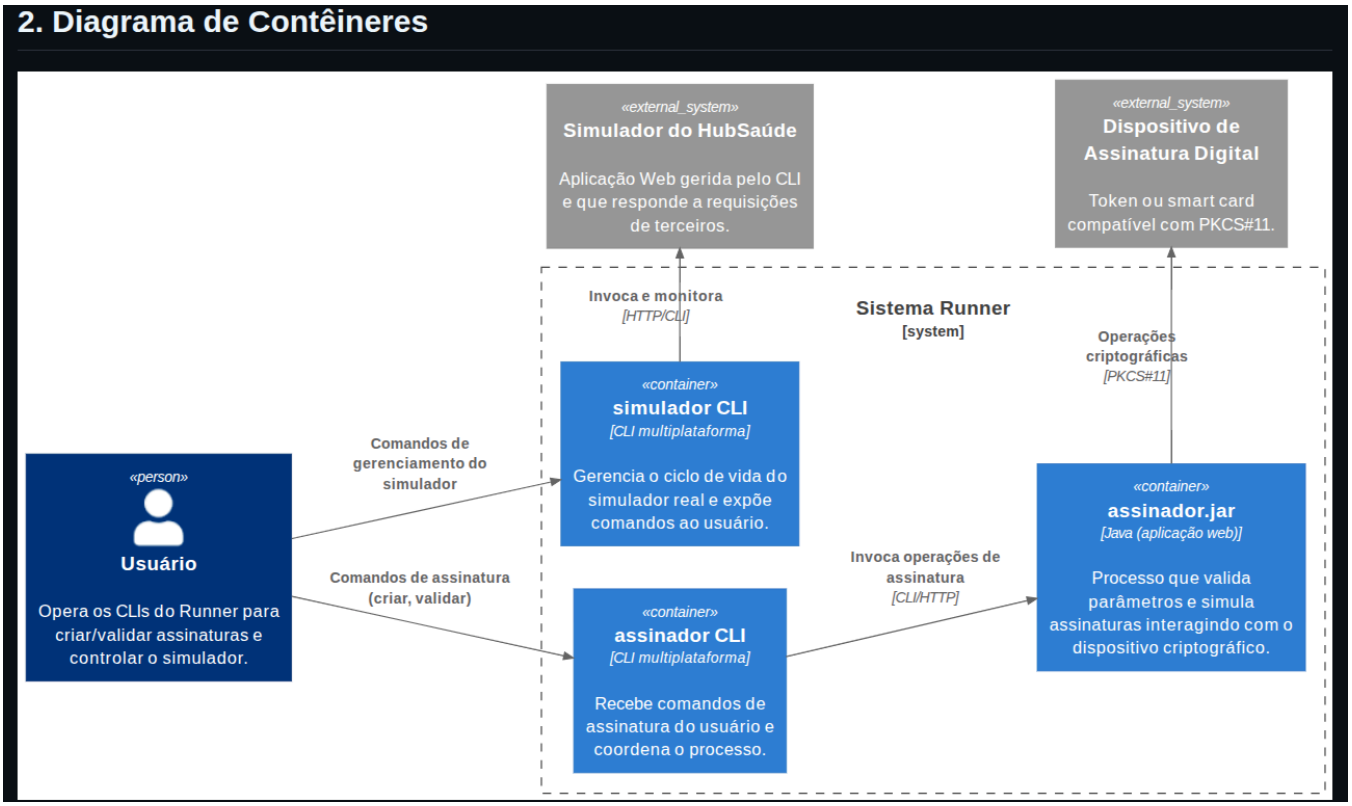

Figura 2 — Diagrama de Contêineres do Sistema Runner.

Contêiner	Tecnologia	Responsabilidade	Comunicação
assinatura	Go / Cobra CLI	Receber comandos sign, validate, version, start/stop/status do assinador e repassar chamadas ao assinador.jar.	CLI com usuário; processo local ou HTTP com assinador.jar.
simulador	Go / Cobra CLI	Iniciar, parar, verificar status e baixar artefatos do Simulador do HubSaúde.	CLI com usuário; HTTP com Simulador; HTTPS com releases.

assinador.jar	Java 21	Validar parâmetros, simular criação/validação de assinatura e expor endpoints HTTP em modo servidor.	Processo local, HTTP e PKCS#11.
Simulador do HubSaúde	Java / aplicação web	Responder a requisições de terceiros e ser gerenciado pelo Runner.	HTTP/HTTPS local, endpoints /api/info e /shutdown.
Dispositivo de Assinatura Digital	Token/smart card	Armazenar material criptográfico e realizar operações via interface padrão.	PKCS#11.

6.3 Visão de decomposição lógica do repositório

A decomposição lógica proposta separa comandos executáveis, módulos internos reutilizáveis e o projeto Java. Essa organização permite que os binários assinatura e simulador compartilhem funcionalidades comuns, como parsing de comandos, invocação de processos, provisionamento de JDK/JRE e download de artefatos.

```
runner/
├── cmd/
│   ├── assinatura/       # binário principal do fluxo de assinatura
│   └── simulador/        # binário para gestão do Simulador do HubSaúde
├── internal/
│   ├── cli/              # parsing e configuração de comandos
│   ├── invoker/          # invocação local e HTTP de aplicações Java
│   ├── jdk/              # detecção e provisionamento de JDK/JRE
│   └── release/          # download e verificação de artefatos
├── assinador/            # projeto Java/Maven do assinador.jar
├── .github/workflows/    # CI/CD de build e release
```

```
├── go.mod
└── go.sum
```

Módulo	Responsabilidade	Motivo arquitetural
cmd/assinatura	Ponto de entrada do CLI de assinatura.	Isola comandos e flags do binário assinatura.
cmd/simulador	Ponto de entrada do CLI de gerenciamento do simulador.	Permite distribuição independente do binário simulador.
internal/cli	Configuração de comandos, parsing de flags e mensagens de ajuda.	Evita duplicação de lógica entre CLIs.
internal/invoker	Execução de java -jar, chamadas HTTP e captura de saída.	Centraliza regras de invocação e tratamento de erro.
internal/jdk	Detecção e provisionamento de JDK/JRE.	Mantém portabilidade e simplifica uso para o usuário final.
internal/release	Download, cache e verificação de artefatos.	Permite atualização dinâmica de simulador.jar e JDK/JRE.
assinador	Projeto Java do assinador.jar.	Mantém a lógica Java no mesmo repositório para CI unificada.
.github/workflows	Automação de testes, build, release, checksums e assinatura.	Garante repetibilidade e distribuição segura.

6.4 Visão de integração e execução

6.4.1 Fluxo de assinatura em modo local

No modo local, cada comando executado pelo usuário inicia uma nova execução do assinador.jar por meio de java -jar. Essa estratégia é simples e adequada para execuções esporádicas, mas tem maior latência por causa da inicialização da JVM a cada chamada.

```
Usuário
  -> assinatura CLI
  -> validação inicial de argumentos
  -> java -jar assinador.jar sign|validate ...
```

```
-> assinador.jar valida parâmetros
-> assinador.jar retorna resultado simulado
-> assinatura CLI formata e exibe a resposta
```

6.4.2 Fluxo de assinatura em modo servidor HTTP

No modo servidor, o assinador.jar permanece em execução aguardando requisições HTTP. O CLI assinatura reutiliza a instância ativa quando possível, evitando inicializações repetidas da JVM e reduzindo latência em chamadas subsequentes.

```
Usuário
  -> assinatura CLI
  -> verifica registro de PID/porta em ~/.hubsaude/
  -> executa health check HTTP
  -> se ativo: envia POST /sign ou POST /validate
  -> se inativo: inicia servidor ou usa fallback local
  -> recebe resposta
  -> formata resultado ao usuário
```

6.4.3 Fluxo de gerenciamento do Simulador do HubSaúde

O CLI simulador gerencia o ciclo de vida do simulador.jar. Antes de iniciar o processo, verifica se a porta padrão está disponível, localiza ou baixa o artefato necessário, registra metadados de execução e fornece comandos para status e encerramento.

```
simulador start
  -> verifica porta 8443
  -> localiza simulador.jar em cache
  -> se ausente/desatualizado: baixa release mais recente
  -> localiza ou provisiona JDK/JRE
  -> inicia processo Java
  -> registra PID/porta em ~/.hubsaude/
  -> exibe status ao usuário
```

6.5 Visão de interfaces

Interface	Origem	Destino	Protocolo/Formato	Observações
Comandos de assinatura	Usuário	assinatura CLI	CLI	Comandos como sign, validate, version e opções relacionadas.
Invocação local do assinador	assinatura CLI	assinador.jar	Processo local / java -jar	Modo simples, com cold start da JVM.
Invocação HTTP do assinador	assinatura CLI	assinador.jar	HTTP / JSON	Modo servidor, usando endpoints como POST /sign e POST /validate.
Operações PKCS#11	assinador.jar	Dispositivo criptográfico	PKCS#11	Integração com token/smart card ou simulador equivalente.
Gerenciamento do simulador	simulador CLI	Simulador do HubSaúde	HTTP	Endpoints previstos: /api/info para status e /shutdown para encerramento.
Download de artefatos	CLIs	GitHub Releases ou release.json	HTTPS	Usado para simulador.jar, binários, checksums e metadados de versão.
Provisionamento Java	CLIs	Adoptium/Temurin	HTTPS	Download de JDK/JRE compatível

				quando ausente.

6.6 Visão de dados e estado local

O Sistema Runner não tem como objetivo persistir assinaturas digitais. Porém, precisa manter dados operacionais locais para cache, controle de processos, versões baixadas e configuração de ambiente. O diretório previsto para isso é ~/.hubsaude/.

Dado/artefato	Local previsto	Finalidade	Persistência
Metadados de processo	~/.hubsaude/	Registrar PID, porta e status esperado de assinador.jar e simulador.jar.	Temporária, atualizada ao iniciar/parar processos.
JDK/JRE gerenciado	~/.hubsaude/jdk/ ou equivalente	Permitir execução das aplicações Java sem instalação manual pelo usuário.	Persistente até atualização ou remoção.
simulador.jar	~/.hubsaude/releases/ ou cache equivalente	Evitar download repetido da mesma versão do simulador.	Persistente, versionado.
assinador.jar	Diretório do projeto ou release local	Aplicação Java invocada pelo CLI assinatura.	Persistente, distribuído com ou obtido pelo projeto.
Checksums	Junto aos artefatos baixados/publicados	Verificar integridade dos binários e jars.	Persistente por versão.
Logs operacionais	~/.hubsaude/logs/ ou saída do terminal	Auxiliar diagnóstico de falhas.	Opcional, conforme implementação.

6.7 Visão de implantação e distribuição

A implantação para usuário final ocorre por binários pré-compilados publicados em GitHub Releases. O build deve gerar artefatos para Windows, Linux e macOS.

O pipeline de release deve injetar a versão da tag nos binários, gerar checksums e publicar os arquivos na release correspondente.

Ambiente	Artefato esperado	Observações arquiteturais
Windows amd64	assinatura-<versão>-windows-amd64.exe; simulador-<versão>-windows-amd64.exe	Exige extensão .exe e cuidado com caminhos Windows.
Linux amd64	assinatura-<versão>-linux-amd64; simulador-<versão>-linux-amd64	Pode exigir permissão de execução após download.
macOS amd64	assinatura-<versão>-darwin-amd64; simulador-<versão>-darwin-amd64	Pode exigir autorização do sistema para execução de binário baixado.
CI/CD	Artefatos de workflow e releases versionadas	Build, testes, checksums e assinatura automatizados.
Máquina do usuário	Diretório ~/.hubsaude/ para cache e metadados	Reduz necessidade de configuração manual.

6.8 Visão de segurança

A segurança da arquitetura concentra-se em três aspectos: delimitar claramente que a assinatura é simulada, proteger a cadeia de distribuição dos artefatos e tratar adequadamente a integração com dispositivos criptográficos compatíveis com PKCS#11.

Aspecto	Risco	Controle arquitetural previsto
Artefatos distribuídos	Usuário baixar binário adulterado.	Publicar checksums SHA256 e assinaturas Cosign junto aos binários.
Execução de processos	Processos duplicados ou processos órfãos.	Registrar PID/porta, executar health check e

		atualizar estado em ~/.hubsaude/.
Download de JDK/JRE e jars	Download incompleto, corrompido ou de versão incorreta.	Verificar versão, usar origem conhecida e validar checksum quando disponível.
PKCS#11	Ausência de token, biblioteca inválida ou erro de dispositivo.	Isolar tratamento no assinador.jar e exibir mensagens claras ao usuário.
Escopo criptográfico	Usuário interpretar a simulação como assinatura real.	Documentação e mensagens devem explicitar que o fluxo é simulado.
HTTP local	Porta ocupada ou serviço não responsivo.	Verificar porta, executar health check e oferecer fallback local quando aplicável.

6.9 Visão de CI/CD e qualidade

A arquitetura de entrega contínua garante que alterações no código sejam verificadas automaticamente e que os artefatos gerados sejam reproduzíveis. Os workflows de build e release devem separar testes, compilação e publicação.

Workflow	Gatilho	Responsabilidades
build.yml	push e pull_request para main	Executar go vet, go test, compilar para plataformas-alvo e disponibilizar artifacts do workflow.
release.yml	push de tags v*	Executar testes, gerar binários versionados, gerar checksums e publicar no GitHub Releases.
Assinatura de artefatos	Durante release	Assinar binários com Cosign e publicar arquivos .sig e .pem quando previsto.

Testes multiplataforma	Build e release	Executar testes em ubuntu-latest, windows-latest e macos-latest.

7. Decisões arquiteturais

As decisões abaixo registram alternativas escolhidas, motivação e consequências. Esse registro auxilia a rastreabilidade e a manutenção futura.

ID	Decisão	Justificativa	Consequências
DA-01	Usar Go para os CLIs assinatura e simulador.	Go facilita cross-compilation, gera binários nativos e atende bem aplicações CLI.	Reduz necessidade de runtime externo para os CLIs, mas exige integração explícita com Java.
DA-02	Usar Java 21 para o assinador.jar.	Restrição do projeto e adequação ao ecossistema Java para aplicações .jar.	Exige JDK/JRE compatível e provisionamento automatizado.
DA-03	Adotar dois modos de invocação do assinador.jar: local e HTTP.	Modo local é simples; modo HTTP reduz latência em múltiplas chamadas.	Aumenta complexidade de gerenciamento de processo e health check.
DA-04	Usar ~/.hubsaude/ para cache e metadados locais.	Centraliza dados operacionais do Runner no espaço do usuário.	Exige cuidado com limpeza, atualização e compatibilidade de paths.
DA-05	Distribuir binários via GitHub Releases.	Facilita acesso a versões estáveis e artefatos por plataforma.	Pipeline deve produzir artefatos corretamente nomeados e versionados.

DA-06	Gerar checksums SHA256 e assinatura Cosign.	Aumenta confiança na integridade e origem dos artefatos.	Requer configuração adicional no workflow de release.
DA-07	Organizar documentação arquitetural por C4 e ISO/IEC/IEEE 42010.	C4 é adequado para explicar contexto e contêineres; a norma organiza stakeholders, visões e correspondências.	Exige manter a documentação sincronizada com a implementação.
DA-08	Manter projeto Go e projeto Java no mesmo repositório.	Permite CI unificada e simplifica rastreabilidade da integração.	O repositório terá múltiplas tecnologias e precisa de organização clara.

8. Correspondências, consistência e rastreabilidade

8.1 Correspondência entre requisitos, visões e decisões

Requisito/Épico	Descrição resumida	Visões relacionadas	Decisões relacionadas
US-01	Invocar assinador.jar via CLI.	Contexto, Contêineres, Integração e Execução, Interfaces.	DA-01, DA-02, DA-03, DA-04.
US-02	Simular assinatura digital com validação de parâmetros.	Contêineres, Integração, Segurança.	DA-02, DA-03.
US-03	Gerenciar ciclo de vida do Simulador do HubSaúde.	Contexto, Contêineres, Implantação, Dados locais.	DA-01, DA-04, DA-05.
US-04	Provisionar JDK automaticamente.	Implantação, Dados locais, Segurança.	DA-02, DA-04.

US-05	Disponibilizar binários multiplataforma.	Implantação, CI/CD, Segurança.	DA-01, DA-05, DA-06.

8.2 Correspondência entre preocupações e visões

Preocupação	Visões que respondem à preocupação
Portabilidade	Implantação e distribuição; CI/CD; Decomposição lógica.
Simplicidade de uso	Contexto; Interfaces; Integração e execução.
Integração CLI–Java	Contêineres; Integração e execução; Interfaces.
Gerenciamento de ciclo de vida	Integração e execução; Dados locais; Segurança.
Provisionamento de ambiente	Implantação; Dados locais; Segurança.
Validação e tratamento de erros	Integração; Interfaces; Qualidade.
Segurança de artefatos	Segurança; CI/CD; Implantação.
Manutenibilidade	Decomposição lógica; Decisões arquiteturais; Rastreabilidade.

8.3 Pontos de consistência e possíveis divergências

- A Sprint 1 define o módulo Go como github.com/kyriosdata/runner, enquanto um critério do plano revisado menciona github.com/kyriosdata/assinatura. Para consistência arquitetural, recomenda-se adotar github.com/kyriosdata/runner, pois representa o sistema completo e não apenas o CLI assinatura.
- O diagrama de contêineres usa o termo assinador CLI em alguns pontos, enquanto a especificação também usa assinatura como nome da aplicação CLI. Recomenda-se padronizar assinatura para o CLI e assinador.jar para a aplicação Java.
- O Simulador do HubSaúde aparece como sistema externo no design, mas a especificação lista o código fonte do simulador como entregável. Recomenda-se confirmar com o professor se o escopo exige implementar o simulador ou apenas gerenciá-lo por meio do CLI.
- Os exemplos de artefatos da especificação usam extensões AppImage e dmg, enquanto as tarefas operacionais da Sprint 1 tratam binários simples. Recomenda-se alinhar a estratégia de empacotamento antes da entrega final.

9. Riscos arquiteturais e pontos de atenção

ID	Risco	Impacto	Mitigação recomendada
RA-01	Diferenças entre sistemas operacionais para gerenciamento de processos.	Falhas em start/stop/status em Windows, Linux ou macOS.	Isolar lógica por plataforma e testar em runners nativos.
RA-02	Download automático de JDK/JRE falhar ou mudar formato.	Usuário não consegue executar aplicações Java.	Tratar erros claramente, permitir configuração manual e validar integridade.
RA-03	Portas ocupadas ou servidor HTTP não responsivo.	Fluxos de servidor falham ou criam instâncias duplicadas.	Verificar porta, registrar PID/porta e usar health check antes de reutilizar.
RA-04	Integração PKCS#11 complexa para ambientes reais.	Atraso na Sprint 3 e dificuldade de testes.	Usar SoftHSM2 ou simulador equivalente nos testes e isolar integração.
RA-05	Confusão entre assinatura simulada e real.	Uso incorreto do sistema em contexto não previsto.	Documentar explicitamente que a assinatura é simulada e não possui validade criptográfica real.
RA-06	Pipeline de release publicar artefatos incompletos.	Usuários baixam binários sem checksum ou assinatura.	Usar job publish dependente de build/test e validar release v0.1.0.
RA-07	Acoplamento excessivo entre CLI e assinador.jar.	Dificuldade de evoluir comandos e modos de invocação.	Definir camada internal/invoker e contratos claros de entrada/saída.

10. Estratégia de verificação e validação da arquitetura

10.1 Verificação por critérios de arquitetura

Critério	Como verificar	Evidência esperada
Portabilidade	Executar testes em Windows, Linux e macOS.	Workflows com matrix multiplataforma aprovados.
Build local	Executar go build ./..., go vet ./... e go test ./....	Comandos finalizados sem erro.
Versionamento	Criar tag v0.1.0 e verificar release.	Binários com versão injetada e checksums publicados.
Invocação local	Executar CLI assinatura chamando java -jar assinador.jar.	Resposta simulada exibida corretamente.
Invocação HTTP	Iniciar assinador.jar em modo servidor e chamar /sign e /validate.	Respostas HTTP válidas e fallback local quando necessário.
Gerenciamento do simulador	Executar simulador start, status e stop.	PID/porta registrados e processo encerrado corretamente.
Segurança de artefatos	Verificar checksum e assinatura Cosign.	Validação de integridade/autenticidade bem-sucedida.

10.2 Cenários de qualidade arquitetural

Atributo	Cenário	Resposta arquitetural esperada
Portabilidade	Usuário baixa o binário adequado para sua plataforma.	O binário executa sem necessidade de compilação local.
Usabilidade	Usuário fornece parâmetro inválido no comando sign.	O CLI ou assinador.jar informa o parâmetro inválido e orienta correção.
Desempenho	Usuário realiza várias assinaturas em sequência.	O CLI reutiliza assinador.jar em modo servidor quando disponível.

Disponibilidade local	Servidor do assinador registrado não responde.	O CLI considera a instância inativa e inicia nova instância ou usa fallback local.
Manutenibilidade	Equipe adiciona novo comando ao CLI.	A estrutura Cobra e internal/cli permite adicionar comando sem alterar módulos não relacionados.
Segurança	Usuário baixa artefato da release.	Usuário consegue verificar SHA256 e assinatura Cosign do arquivo.

11. Glossário

Termo	Definição
CLI	Command Line Interface, interface de linha de comandos usada pelo usuário no terminal.
JAR	Java Archive, formato de empacotamento de aplicações Java.
JDK	Java Development Kit, ambiente completo para desenvolvimento e execução de aplicações Java.
JRE	Java Runtime Environment, ambiente necessário para executar aplicações Java.
PKCS#11	Interface padrão para comunicação com dispositivos criptográficos, como tokens e smart cards.
C4 Model	Modelo de documentação arquitetural que organiza diagramas em níveis como contexto, contêineres, componentes e código.
Viewpoint	Ponto de vista arquitetural que define preocupações e modelos usados para construir uma visão.
View	Visão arquitetural produzida a partir de um ponto de vista para responder a preocupações de stakeholders.

Cosign	Ferramenta do ecossistema Sigstore usada para assinatura e verificação de artefatos.
Checksum SHA256	Resumo criptográfico usado para verificar integridade de arquivos.
Cold start	Inicialização completa de uma aplicação a cada chamada.
Warm start	Reutilização de aplicação já inicializada, geralmente com menor latência.

12. Referências

- ISO/IEC/IEEE 42010:2022 — Software, systems and enterprise — Architecture description. Estrutura conceitual usada para organizar stakeholders, preocupações, pontos de vista, visões, modelos, decisões e correspondências.
- Arquivo especificacao.md — Sistema Runner - Trabalho Prático.
- Arquivo design.md — Sistema Runner - Design.
- Arquivo plano-revisitado-v2.md — Plano Revisado #2.
- Arquivo sprint-1-tasks.md — Sprint 1: Tarefas Operacionais.
- Diagramas de Contexto e de Contêineres anexados à solicitação.

Apêndice A — Síntese da arquitetura proposta

A arquitetura do Sistema Runner é uma arquitetura de integração baseada em CLIs multiplataforma que encapsulam a execução de aplicações Java. O sistema privilegia portabilidade, simplicidade de uso, automação de distribuição e separação de responsabilidades. A solução adota Go para binários nativos, Java para o assinador.jar, integração local e HTTP, estado operacional em diretório do usuário e CI/CD para geração segura de releases.

A principal vantagem dessa arquitetura é reduzir a complexidade percebida pelo usuário: comandos simples no terminal substituem configurações manuais de Java, execução direta de arquivos .jar, gerenciamento de portas e controle de processos. A principal complexidade técnica desloca-se para os módulos internos de invocação, provisionamento, controle de ciclo de vida e verificação de artefatos.